The GIL, Not the Syscall, Governs Scaling: A Python `io_uring` Runtime with C-Side Completion Reaping

Marimuthu Velayutham*

October 6, 2026

Abstract

Python event loops built on Linux `io_uring` report throughput gains over `asyncio` and attribute them to system-call amortization: one `io_uring_enter` reaps many completions. We separate that explanation from a second one—where the per-event loop runs relative to CPython's Global Interpreter Lock (GIL)—with a 2×2 experiment: one server loop implemented on `io_uring` and on `epoll`, each in C with the GIL released and in Python. With one worker both factors matter and are of similar size: under a C loop `io_uring` is 1.16× faster than `epoll`, and batching completions is worth about 20%, all of it obtained by 16 completions per call. Scaling across worker threads follows the other factor. With four worker threads in one interpreter on a four-core server, both C loops scale (2.24× and 2.34×) and both Python loops do not (0.96× and 0.64×). We present `uringpy`, a runtime whose workers reap completions and run the accept/receive/send protocol inside a single `nogil` C region. On a standard CPython 3.14 build its worker threads perform as worker processes do (328 thousand requests per second at four workers, 2.33× one worker), while `asyncio` and `uvloop` threads fall to 0.36–0.59× of one worker; it is 1.33× faster than the best process-per-worker baseline. On the free-threaded build `asyncio` threads scale again (2.50–2.64×) and the runtime's advantage at four workers is a factor of 1.5–2.0× in speed, not a difference in scaling. When every request calls a Python handler, thread scaling under the GIL stops at 1.57×, whereas worker processes and free-threaded threads reach about 2.3×. Every configuration is measured in five interleaved repetitions and reported with 95% confidence intervals; the code, the raw runs and the script that generates the tables are public. **[[PENDING: Abstract, introduction and conclusion quote the Python-loop results (0.96×, 0.64×) and the handler comparison. Both are being re-measured: `py-uring` with the GIL released during submission, and the handler workload against Protocol-API and `uvloop` baselines. Revise these sentences, and the title if necessary, when the results are in.]]**

Index Terms— io_uring, asynchronous I/O, Python, Global Interpreter Lock, event loop, multicore scaling, system-call amortization

*M. Velayutham is an independent researcher (e-mail: marimuthu.vm@gmail.com; ORCID: 0009-0007-7352-5552).

1 Introduction

Completion-based kernel interfaces such as Linux `io_uring` [1] let an application submit many I/O operations and reap many completions with a single system call, in contrast to readiness-based interfaces (`epoll`), which report which sockets are ready and leave the application to make one system call for every read and every write. A growing set of Python projects wrap `io_uring` behind an `asyncio`-compatible event loop and report throughput improvements over the standard library loop and over `uvloop` [2], generally crediting the win to system-call amortization.

Two explanations for such a win are possible, and they are rarely separated. One is the interface: fewer system calls per request. The other is where the per-event loop runs. CPython serializes bytecode execution behind a Global Interpreter Lock (GIL), so a loop that dispatches every event in Python pays an interpreter cost per event and, when several worker threads run such loops, lets only one of them proceed at a time. A faster interface does nothing about that cost, and a loop moved out of the interpreter would be faster on any interface. Crediting a measured gain to `io_uring` therefore needs an experiment in which the two are varied independently.

We run that experiment. One server loop is implemented four times: on `io_uring` and on `epoll`, each in C with the GIL released and in Python. The answer has two parts. For a single worker both factors matter and are of similar size: `io_uring` is 1.16× faster than `epoll` under a C loop, and restricting it to one completion per system call costs 20%. For scaling, the placement of the loop is what separates the designs: four C-loop workers in one interpreter reach 2.24× and 2.34× the throughput of one on the two interfaces, while four Python-loop workers reach 0.96× and 0.64×. The system call sets how fast one worker is; the GIL decides whether a second worker helps. That is the sense of the title.

Guided by this result, we build a runtime, `uringpy`, whose worker performs completion reaping *and* the full accept/receive/send state machine inside one `nogil` region, creating no Python object per completion. N such workers, each with its own ring and its own `SO_REUSEPORT`

listener, scale across cores inside a single interpreter on a standard GIL build. The `asyncio`-compatible `io_uring` loops we are aware of dispatch each event to Python and so cannot. We also measure where the advantage ends: on the free-threaded build, where `asyncio` threads scale again; when every request runs a Python handler; and when responses are large enough for the network to be the limit.

Contributions.

- A cost model and two controlled experiments—a contention probe, and a 2×2 design that crosses the system-call interface with the placement of the loop—showing that the interface sets single-worker throughput while the time a request spends under the GIL governs scaling across worker threads (Sections 3 and 5).
- A GIL-aware runtime that reaps completions and runs the connection protocol in C, enabling single-interpreter multicore scaling (Section 4).
- An evaluation across the thread/process model × GIL/free-threaded build matrix—spanning transport-only and application-handler workloads and batching and response-size regimes, with confidence intervals and processor accounting—including negative and boundary results (Section 5).

2 Background and Related Work

Amortizing system calls. The cost of crossing the user/kernel boundary once per I/O operation has long been a target. FlexSC [3] showed that batching system calls and executing them asynchronously removes much of the direct and indirect cost of mode switches. MegaPipe [4] combined system-call batching with partitioned listening sockets and lightweight connection handles, and user-level or dataplane stacks such as mTCP [5] and IX [6] batch events and packets aggressively. `io_uring` [1] brings the same idea into the mainline Linux interface: a submission ring and a completion ring are shared between user space and the kernel, and one `io_uring_enter` call submits many operations and reaps many completions. Studies of `io_uring` have so far concentrated on storage and database engines. Didona et al. [7] compare it with `libaio` and SPDK and find that its performance depends strongly on how it is configured, and Jasny et al. [8] report that substituting it for existing I/O calls gains little in a database system, whereas a design built around its asynchronous, batched model gains substantially. Both results warn against attributing a speed-up to the interface alone; this paper asks the same question for a managed-language network server, where a second cost—the interpreter—is present on every event.

Event-driven and shared-nothing servers. Event-driven servers built on readiness notification [9,10] are the model that `asyncio` inherits [11]. Scaling such servers across cores is usually done by giving each core its own event loop and its own accept queue. Affinity-Accept [12] showed the benefit of per-core accept queues for connection locality, `SO_REUSEPORT` [13] made the kernel distribute connections across several listening sockets, and thread-per-core designs have been studied for their effect on tail latency [14]. Our workers follow this structure: one ring and one `SO_REUSEPORT` listener per worker, with no state shared between them.

The GIL. CPython serializes bytecode execution behind a Global Interpreter Lock. Beazley [15] characterized how hand-off of the lock between threads behaves, including the degradation that I/O-bound threads suffer when other threads compete for it. Three lines of work aim to remove the limitation. The lock can be made unnecessary inside the interpreter, as explored for Ruby with hardware transactional memory [16] and now delivered for CPython by the free-threaded build [17], which is a supported configuration from Python 3.14 [18]. The lock can be multiplied: each subinterpreter has had its own GIL since Python 3.12 [19], with a standard-library interface in 3.14 [20]. Or the work can be moved out from under the lock, into native code that releases it; Cython [21] makes this convenient through `nogil` regions, and it is the approach taken here. On free-threaded builds the standard event loop itself has been reworked to scale across threads [22], which is why we evaluate against CPython 3.14 with and without the GIL. Whatever part of a request still runs under the lock bounds thread scaling in the manner of Amdahl's argument [23]; Section 3 makes this bound explicit.

Python servers and event loops. `uvloop` [2] reimplements the `asyncio` loop over libuv. Several projects (e.g. `uringloop` [24], `uringcore` [25]) provide `io_uring`-backed loops and attribute their gains over `asyncio` to fewer system calls; like `asyncio`, they dispatch every event to Python code. Moving network I/O out of the interpreter is itself established practice: Granian [26], for example, performs it on Rust threads built on Hyper and Tokio and enters Python only to run the application. Our runtime applies that principle to `io_uring` completion reaping, which the existing `io_uring` loops for Python do not, and so scales worker threads across cores within one interpreter. It is also deliberately small enough to be paired with an otherwise identical `epoll` reactor and with Python-dispatch loops on each interface. That pairing supplies what has been missing: a measurement that separates the two candidate explanations for the speed of such designs—the system-call interface and the placement of the per-event loop—and that measures what happens to thread scaling in one interpreter once every request enters Python.

Measurement practice. Performance comparisons are easily distorted by too few repetitions and by uncontrolled properties of the setup [27,28], and cloud networks add variability of their own [29]. We therefore repeat ev-

ery configuration, interleave the repetitions, and report means with confidence intervals together with CPU use on both machines. Load is generated with `wrk` [30], a closed-loop generator; its latency figures describe latency at the offered load and are subject to coordinated omission [31], so we use them only to compare configurations at the same load, not as tail-latency estimates.

3 Isolating the Bottleneck

Hypothesis. We began from a broader hypothesis: that for a Python `io_uring` server, where the per-event loop runs matters more than which system-call interface is used. Section 5 shows that this fails for a single worker, where the two matter about equally. What the data supports is narrower, and we state it here as the claim the rest of the paper examines: whether worker threads scale is governed by what each request does under the GIL, while the interface affects how fast a single worker is.

Cost model. Let a request cost t_c of processor time that needs no interpreter state, and so can run with the GIL released, and t_p that must run with the GIL held. A saturated single worker serves $X_1 = 1/(t_c + t_p)$ requests per second. With N worker threads in one interpreter on at least N cores, two constraints apply. Each worker still needs $t_c + t_p$ per request, so $X_N \le N/(t_c + t_p)$. And only one thread holds the GIL at a time, so the GIL is busy for $X_N\, t_p \le 1$ of every second, that is, $X_N \le 1/t_p$. Dividing by X_1 gives the scaling bound

$$S(N) = \frac{X_N}{X_1} \le \min\left(N,\ \frac{1}{f}\right), \qquad f = \frac{t_p}{t_c + t_p}, \qquad (1)$$

where f is the fraction of a request spent holding the GIL. Its two limiting cases are the two designs compared in this paper. A loop that holds the GIL for all of a request's processor time has $f = 1$ and cannot exceed one worker's throughput however many threads it runs. (A loop written in Python need not be at that limit: system calls made with the GIL released count towards t_c.) A loop that never enters the interpreter has $f = 0$ and is bounded only by N. Between them, a C reactor that calls a Python handler per request has f set by the handler, and (1) states how much handler work a given number of threads can absorb.

Three remarks. First, (1) is an upper bound and may be far from tight: handing the GIL from one thread to another is not free, and a thread that wants the GIL waits for it. Models that include contention for critical sections give lower values [32]. We therefore measure, per request, both the time the GIL is held (t_p) and the time spent waiting to acquire it (w); w is what separates measured scaling from the bound. Second, with one process per worker each worker has its own GIL and the second constraint disappears, leaving $S(N) \le N$ for any f; process-mode results are thus the reference for what the hardware allows. Third, the system-call interface does not appear in (1). It changes t_c, and hence single-worker throughput, but not which constraint binds. That is the precise sense in which we claim that the GIL, not the system call, governs scaling; Section 5 tests the claim by varying the interface and the placement of the loop independently. **[[PENDING: Handler sweep running: report the measured t_p, w and predicted-against-measured scaling here and in Section 5.]]**

Table 1: Contention probe: time to process a fixed number of events with K threads competing for the GIL, relative to $K = 0$.

K	Python loop	C `nogil` loop
0	1.00×	1.00×
1	2.04×	1.02×
2	2.08×	1.02×
3	2.18×	1.03×

Method. We process a fixed number of synthetic completions two ways with identical per-completion arithmetic (a `splitmix64` step), while K CPU-bound Python threads contend for the GIL:

- *reactor_python*: an interpreted per-completion loop (GIL held), modeling per-event Python dispatch as in `asyncio`.
- *reactor_nogil*: the same arithmetic inside one C `nogil` region (GIL released), modeling C-side batch reaping.

Each design is auto-calibrated to run ≈0.25 s at rest, so both are measured over comparable, stable durations. We report each design's *contention slowdown* $= t(K)/t(0)$.

Result. Table 1 shows that the interpreted loop takes about twice as long as soon as one thread competes for the GIL (2.04×) and stays near that level with two and three competitors (2.08×, 2.18×), while the C loop remains within 3% of its uncontended time. The probe performs no I/O and makes no system call. It therefore shows that the placement of the loop matters under contention, and nothing about the interface; Section 5 varies the interface. (Environment: the server of Table 2, CPython 3.14.7, GIL build; $K \le 3$ so that every thread has a core of its own.) **[[PENDING: Table 1 is the best of five timings per cell. Re-run with the updated script (mean and standard deviation over 20 repetitions) once the server is idle, and replace the numbers.]]**

4 Design

`uringpy` is a Cython [21] extension over `liburing`. Its unit is the *engine*: one `io_uring` instance, one pool of fixed-size receive buffers, and a reactor loop. A server is N engines, each driven by its own OS thread and each with its own `SO_REUSEPORT` listening socket on the same port, so the kernel spreads incoming connections over the workers (Fig. 1). Engines share nothing in user space:

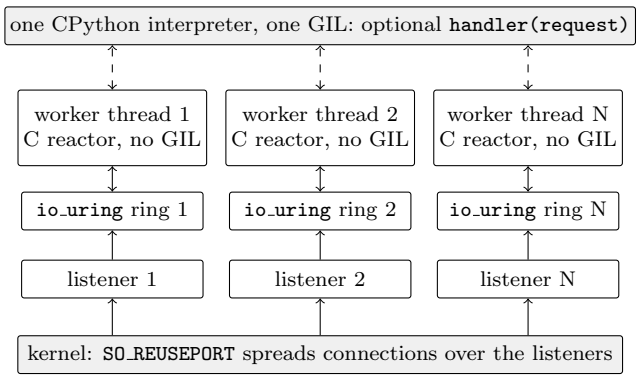


Figure 1: A server is N engines in one process. Solid arrows are the data path, which never leaves C; the dashed arrow is the only point where a worker takes the GIL, and exists only in application mode.

no ring, buffer or connection is touched by two workers, so the data path takes no user-level lock. Each ring is created with default flags and uses single-shot accept, receive and send operations, without registered buffers or files; the interface effects reported in Section 5 are for this basic use of `io_uring`.

The loop. Fig. 2 gives the loop each worker runs; all of it executes inside one `nogil` region. Each iteration makes exactly one system call, `io_uring_submit_and_wait`, which hands the kernel every operation queued while the previous batch was processed and returns when at least one completion is ready. The loop then processes every completion already in the ring without re-entering the kernel. Follow-up operations (the next receive, the send of a response) are written into the submission ring and leave with the next iteration's call. No Python object is created for a completion, and the interpreter is not entered.

Per-operation state. Each submitted operation carries a 64-bit tag that the kernel returns with its completion: 8 bits of operation type, 32 bits of file descriptor, and 24 bits that hold the receive-buffer slot for a receive or the number of response bytes already sent for a send. Dispatch therefore needs no lookup table. Because the sent offset travels with the operation, a send that the kernel completes only in part is continued from where it stopped, and a response may be larger than a buffer slot (up to 16 MiB). Receive buffers come from a per-engine slab allocator; a connection for which no buffer or submission entry is available is closed, not left idle.

Two entry points. `serve_forever_echo` answers every received request with a fixed response and never takes the GIL; it measures the transport path alone. `serve_forever_app` passes each request to a Python callable. The reactor takes the GIL for that call only: it builds the request `bytes`, invokes the handler, copies the result into a per-connection buffer owned by the engine, and releases the GIL before the send is queued. Accept, receive and send stay outside the GIL in both. Neither

```
1:  queue ACCEPT on the listener
2:  while not stopped do
3:    io_uring_submit_and_wait(1) {one system call}
4:    for each completion c in the ring, up to the batch
      cap do
5:      if c is an ACCEPT then
6:        queue ACCEPT; queue RECV on the new con-
          nection
7:      else if c is a RECV of n > 0 bytes then
8:        if application mode then
9:          take GIL; r ← handler(request)
10:         copy r to the connection's buffer; release
            GIL
11:       end if
12:       queue SEND of the response from offset 0
13:     else if c is a SEND of n > 0 bytes then
14:       if part of the response is still unsent then
15:         queue SEND of the remainder
16:       else
17:         count one request; queue RECV
18:       end if
19:     else
20:       close the connection {peer closed, or error}
21:     end if
22:   end for
23: end while
```

Figure 2: The reactor loop of one worker. Queued operations reach the kernel at the next execution of line 3.

entry point parses HTTP: each replies once per `recv`, so pipelined and fragmented requests are out of scope. The same holds for every baseline we compare against.

Instrumentation. The reactor counts its `io_uring_enter` calls, its completions and the responses it has sent in full, so system calls per request and completions per system call are measured, not estimated. A batch cap limits the completions processed per system call, to vary batching in isolation. Optional timing records, per handler call, how long the worker waited for the GIL and how long it held it. The counters are plain integer increments; the timing is disabled unless requested.

A readiness-based twin. For the comparison in Section 5 the package also contains `EpollEngine`, the same loop written against `epoll`: one `recv` and one `send` system call per request, the same reply-once-per-`recv` behaviour and resumption of partial sends, likewise inside one `nogil` region, with every system call counted.

Interpreter integration. A wait that a signal interrupts returns control to the interpreter long enough to run pending Python signal handlers, so a worker on the main thread can be stopped normally. The extension declares itself safe for free-threaded builds, so loading it there does not re-enable the GIL; the benchmark image verifies this when it is built.

Table 2: Testbed.

Server	Google Compute Engine `t2d-standard-4`: 4 vCPUs (AMD EPYC 7B13), one per physical core, 16 GB
Client	`t2d-standard-8`: 8 vCPUs (AMD EPYC 7B13), one per physical core; same zone (`us-east1-b`)
System	Ubuntu 24.04 LTS, Linux 7.0.0-1011-gcp on both
Runtime	Docker, Debian 12 base, `liburing` 2.6; CPython 3.14.7 built from source with and without `--disable-gil`
Load	`wrk` 4.1.0, 8 threads, 400 keep-alive connections, `GET /`; 13-byte response body unless stated
Run	fresh server process; 5 s warm-up (discarded), 20 s measured
Repetitions	5 per configuration, interleaved in shuffled order

5 Evaluation

5.1 Setup

Table 2 lists the testbed. The server and the load generator are separate virtual machines in one zone. Both are of a machine family in which each vCPU is a whole physical core [33], so a worker count equals a core count, and the client has twice the server's cores so that it is not the limit. The code under test runs in a container on the server with host networking. The two interpreters are built from one recipe that differs only in `--disable-gil`, so a comparison between them is not also a comparison of compilers or build options, and every run records the interpreter version and the GIL state the server process reports.

5.2 Engines

Table 3 lists the servers compared. All of them answer each `recv` with the same bytes and none parses HTTP, so they differ only in how the I/O loop is built. The first four form a 2×2 design that varies the system-call interface and the placement of the per-event loop independently. Each engine runs as N workers with one `SO_REUSEPORT` listener per worker, either as threads of one interpreter (*thread* mode, one GIL) or as separate processes (*process* mode, one GIL each). The two Python loops of the 2×2 design are bare dispatch loops written directly on the interface, with none of the task, future and transport machinery of `asyncio`; they isolate the cost of running the loop in the interpreter and are not event loops one would deploy.

5.3 Metrics

For a configuration with n runs and throughputs $x_1, \ldots, x_n$ (requests per second over the measured interval) we report the mean $\bar{x}$ with the half-width of its 95% confidence interval,

$$h = t_{0.975,\,n-1}\,\frac{s}{\sqrt{n}}, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \quad (2)$$

where t is the Student-t quantile (2.776 for $n = 5$). Scaling at N workers is the ratio to the one-worker configuration of the same engine and mode, with a half-width from first-order error propagation:

$$S_N = \frac{\bar{x}_N}{\bar{x}_1}, \qquad h_S = t\,S_N\sqrt{\frac{s_N^2}{n_N\,\bar{x}_N^2} + \frac{s_1^2}{n_1\,\bar{x}_1^2}}, \quad (3)$$

with t taken at $\min(n_N, n_1) - 1$ degrees of freedom.

Processor use is sampled from `/proc/stat` on both machines immediately before and after each measured interval. With u the busy share of a machine with c cores, $C = c\,u$ is the number of busy cores, and we report throughput per busy core, $\bar{x}/C$, next to S_N: a one-worker server also uses other cores for kernel network processing, so S_N alone understates per-core scaling. We also give this quantity relative to one worker, $E_N = (\bar{x}_N/C_N)/(\bar{x}_1/C_1)$, so that $S_N = E_N\,C_N/C_1$. A configuration is flagged when the client's busy share exceeds 85%, since the load generator may then be the limit.

From the reactor's counters we report system calls per request and completions per `io_uring_enter`, each as a ratio of counters summed over a configuration's runs, and, in application mode, the GIL hold time t_p and wait time w per handler call. Latency percentiles are those of `wrk` and are used only to compare configurations at equal load (Section 2).

Table 3: Engines. "C" means the loop runs with the GIL released.

Engine	Interface	Per-event loop
`uringpy`	`io_uring`	C
`c-epoll`	`epoll`	C (same loop)
`py-uring`	`io_uring`	Python
`py-epoll`	`epoll`	Python
`asyncio`	`epoll`	Python, streams API
`asyncio-proto`	`epoll`	Python, Protocol API
`uvloop(-proto)`	libuv	C loop, Python callbacks
`uringpy-app`	`io_uring`	C + Python handler
`asyncio-app`	`epoll`	Python, same handler

5.4 Criterion

The broader hypothesis of Section 3 was given a test. It is supported if, in the 2×2 design, moving the loop from Python to C changes throughput far more than switching between `epoll` and `io_uring`, and if capping the completions processed per system call changes throughput little. Otherwise the paper reports what the data shows. We do not present this as a pre-registered test: the criterion was written down on the day of the measurements, while the 2×2 runs were in progress, and the commit that records it is later than those runs.

5.5 Results

The result tables are generated from the raw runs by a script in the repository (`benchmarks/make_tables.py`); no number in them is transcribed by hand. They cover 550 runs of 110 configurations. No run failed and none reported a socket error or a non-2xx response. Hypervisor steal time was zero on both machines throughout, and the client's busy share, averaged over a configuration's runs, never exceeded 78% (80% in any single run), below the 85% threshold of Section 5.3. **[[PENDING: Update the counts when the handler sweep is added.]]**

Interface and loop placement. Table 4 gives the 2×2 design and Table 5 the effect of each factor with the other held fixed. The counters confirm that the design varies what it is meant to vary: the `epoll` loops make 2.0 system calls per request (one `recv` and one `send`), the `io_uring` loops between 0.006 and 0.04.

With one worker, both factors matter and neither dominates. Changing the interface from `epoll` to `io_uring` raises throughput by a factor of 1.16 ± 0.06 when the loop is in C and 1.26 ± 0.03 when it is in Python. Moving the loop from Python to C raises it by 1.10 ± 0.04 on `io_uring` and 1.20 ± 0.04 on `epoll`. The second pair is a lower bound on what a full event loop pays, because the Python loops here are bare: `asyncio`'s streams API, which like `py-epoll` runs on `epoll`, reaches 66.8 thousand requests per second on one worker (Table 7) against 105.7 for `py-epoll`.

With more than one worker thread the two C loops and the two Python loops part ways. Both C loops scale, to $S_4 = 2.24 \pm 0.08$ on `io_uring` and 2.34 ± 0.07 on `epoll`. Neither Python loop does, but they fail differently. `py-epoll` gains at two workers (1.33 ± 0.04) and falls below one worker at four (0.64 ± 0.02). `py-uring` stays at the throughput of one worker ($S_2 = 0.97$, $S_4 = 0.96$) with only 1.52 of the four cores busy, and `wrk` reports a 99th-percentile latency near one second at two and at four workers (3.2 ms at one), while the median falls from 3.0 ms at one worker to 0.9 ms at four: some connections are served quickly and others starve.

The two Python loops differ in more than the interface, and the difference matters for reading these rows. `py-epoll` makes its `recv` and `send` calls through CPython's socket objects, which release the GIL around each system call. `py-uring` releases the GIL while it waits for completions but, as measured here, holds it while submitting, and the kernel performs the queued sends during that call. In the terms of (1), `py-uring` as measured has f close to 1 because of where it holds the GIL, not because its dispatch is interpreted, and its flat scaling cannot be attributed to the placement of the loop alone. **[[PENDING: Re-measure `py-uring` with the GIL released during submission (the 2x2 rerun), report both variants, and rewrite this paragraph and the sentence below according to the outcome. If the corrected loop scales, batching helps scaling by amortizing GIL hand-offs and the claim must be narrowed further.]]** At four workers the C loop is 2.55× (`io_uring`) and 4.41× (`epoll`) faster than the Python loop on the same interface, while the interface effect is 1.11× under a C loop and 1.91× under a Python loop.

Batching. Table 6 varies batching alone. Capped at one completion per call, `uringpy` makes 2.0 system calls per request, the same number as an `epoll` loop, and delivers 0.80 ± 0.03 of its uncapped throughput. A cap of 4 gives 0.93 ± 0.04, and from a cap of 16 upward throughput cannot be distinguished from the uncapped run, although system calls per request go on falling twentyfold, from 0.126 to 0.006. On this workload, then, batching is worth about 20%, and all of that is obtained by 16 completions per call. An uncapped single worker reaches 330–370 completions per call across our experiments, and each of four workers, among which the 400 connections are divided, about 90 (0.006 and 0.022 system calls per request in Table 4); both are far beyond what throughput requires. Capped at one, `io_uring` is somewhat slower than the `epoll` loop (115.3 against 126.4 thousand requests per second, measured in two different experiments): without batching, the completion interface has no advantage here.

Outcome of the criterion. The criterion of Section 5.4 is met in part, so the broader hypothesis is not supported as stated. Its first condition, that moving the loop changes throughput far more than changing the interface, holds at four workers (2.55–4.41× against 1.11× under a C loop) and fails at one (1.10–1.20× against 1.16–1.26×). Its second condition, that capping the batch changes throughput little, holds for caps of 16 and above and fails below. We therefore do not claim that the system call is unimportant. The narrower claim was formulated after seeing these results and should be read as such: the interface, and batching up to a small batch size, set the throughput of one worker; with a loop in C that releases the GIL, worker threads scale on either interface; and the Python loops measured here do not scale on either. The interface is not irrelevant to how the Python loops fail, as the two rows above show.

Scaling across cores. Table 7 compares `uringpy` with the `asyncio` and `uvloop` baselines on the transport path. Three observations follow.

First, `uringpy` worker threads perform as worker processes do: at four workers the ratio of the two is 0.998 ± 0.004. On this path one interpreter with four threads delivers what four interpreters deliver. That is what one should expect of a path that executes no Python, and the measurement confirms that nothing else in the runtime or the interpreter gets in the way; it says nothing yet about requests that do run Python, which the handler workload below addresses.

Second, the scaling it reaches, $S_4 = 2.33 \pm 0.05$, is what this server allows, not a limit of the runtime. One worker already keeps $C_1 = 1.18$ cores busy, because kernel network processing for its connections runs partly on other cores, and four workers keep 3.91 of the four cores busy.

Table 4: The 2×2 design (worker threads, GIL build, 13-byte body): throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), scaling, measured system calls per request at one and four workers, busy server cores and 99th-percentile latency at four workers.

Engine	Interface	Loop	N=1	N=2	N=4	S_2	S_4	sys/req$_1$	sys/req$_4$	C_4	p99$_4$ (ms)
`uringpy`	`io_uring`	C	146.1 ± 5.4	231.5 ± 5.9	327.9 ± 1.2	1.59 ± 0.07	2.24 ± 0.08	0.006	0.022	3.91	1.8
`c-epoll`	`epoll`	C	126.4 ± 3.8	201.7 ± 5.5	296.4 ± 0.7	1.60 ± 0.06	2.34 ± 0.07	2.004	2.014	3.90	2.8
`py-uring`	`io_uring`	Python	133.3 ± 2.1	129.7 ± 1.8	128.3 ± 0.5	0.97 ± 0.02	0.96 ± 0.02	0.011	0.040	1.52	953.5
`py-epoll`	`epoll`	Python	105.7 ± 2.4	140.3 ± 2.2	67.3 ± 1.2	1.33 ± 0.04	0.64 ± 0.02	2.004	2.012	2.46	11.7

Table 5: Effect of each factor with the other held fixed, as a ratio of mean throughputs (± 95% half-width) from Table 4. Interface: `io_uring` over `epoll`. Loop: C over Python.

Effect	N=1	N=2	N=4
Interface, loop in C	1.16 ± 0.06	1.15 ± 0.04	1.11 ± <0.01
Interface, loop in Python	1.26 ± 0.03	0.92 ± 0.02	1.91 ± 0.04
Loop, on `io_uring`	1.10 ± 0.04	1.78 ± 0.05	2.55 ± 0.01
Loop, on `epoll`	1.20 ± 0.04	1.44 ± 0.05	4.41 ± 0.08

Table 6: Batch cap (`uringpy`, one worker): completions processed per `io_uring_enter` limited to the cap. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), ratio to the uncapped run, measured completions per call and system calls per request.

Cap	Throughput	vs. none	compl./call	sys/req
1	115.3 ± 3.4	0.80 ± 0.03	1.0	2.001
4	134.2 ± 4.4	0.93 ± 0.04	4.0	0.501
16	143.3 ± 2.5	1.00 ± 0.04	15.9	0.126
64	141.4 ± 4.5	0.98 ± 0.04	63.0	0.032
256	142.7 ± 2.3	0.99 ± 0.03	241.9	0.008
none	144.0 ± 4.5	1	356.1	0.006

Per busy core, throughput at four workers is $E_4 = 0.70$ of its value at one. The baselines in process mode, which share no lock at all, show the same loss (E_4 between 0.72 and 0.74) and scale by 2.50–2.67×. The loss is therefore a property of the testbed—four cores shared between the workers and the kernel's network stack—and process mode is the reference for what the hardware allows (Section 3). Thread-mode `uringpy` reaches that reference.

Third, every baseline loses throughput when given more threads. At four worker threads `asyncio` delivers 0.52× and 0.48× of one worker through its two APIs, and `uvloop` 0.59× and 0.36×, with 1.6 to 2.2 of the four cores busy and a 99th-percentile latency of 15–27 ms against 2.7–4.2 ms for the same engines in process mode. A loop in C is not sufficient: `uvloop`'s loop is libuv, but it calls back into Python for every event, and it scales no better than `asyncio`.

In absolute terms, one `uringpy` worker is 1.44× faster than the fastest baseline worker (`uvloop-proto`), 1.50× faster than `asyncio-proto` and 2.11× faster than `asyncio`. At four workers the comparison that matters is with the baselines run the way they have to be run under the GIL, as processes: `uringpy` threads are 1.33× faster than `uvloop-proto`, 1.40× faster than `asyncio-proto` and 1.83× faster than `asyncio`. Against the baselines' threads, a configuration nobody would deploy, the factor is 7 to 10. It should be kept in mind that this workload favours the runtime as much as any can: the baselines execute Python for every event and `uringpy` executes none. (`uvloop` was measured in a separate experiment; see Section 7 on comparisons across experiments.)

Free-threaded build. The lower part of Table 7 repeats the experiment without the GIL. `asyncio` threads now scale, by 2.64 ± 0.08 and 2.50 ± 0.05, within 2–4% of the same engines in process mode. The collapse in the upper part of the table is thus an effect of the GIL and not of `asyncio`'s design, and removing the GIL removes it. The price `asyncio` pays for the free-threaded build on one worker is small, 4–5%. `uringpy` is unaffected by the build, as expected of a loop that does not enter the interpreter: its throughput at four workers on the two builds differs by a factor of 0.996 ± 0.005. What remains of its advantage is a difference in speed that narrows slightly as workers are added: 1.58× over `asyncio-proto` and 2.25× over `asyncio` at one worker, 1.46× and 1.96× at four. On the free-threaded build the contribution of C-side reaping is therefore speed per worker, not scaling.

Application handler. In Table 8 every request calls a Python function that parses the request line, computes a checksum of the path and builds the response. On one worker the handler costs `uringpy` 13% of its transport-only throughput in Table 7. Under the GIL, worker threads then stop scaling after the second: $S_2 = 1.55 \pm 0.06$ and $S_4 = 1.57 \pm 0.07$, with 3.25 of the four cores busy. Worker processes scale as on the transport path ($S_4 = 2.36 \pm 0.07$), so at four workers threads deliver 0.66 ± 0.02 of what processes deliver.

The bound (1) does not account for this. One worker needs $1/140.7\,\text{k} = 7.1$ μs per request without the handler and $1/122.3\,\text{k} = 8.2$ μs with it, so the handler adds about 1.1 μs, $f \approx 0.13$ and $1/f \approx 7.6$: the bound allows more scaling than four cores can deliver. Even if all of the added time is spent holding the GIL, at the plateau the GIL is held for only about a fifth of each second. What the bound leaves out is the cost of acquiring a contended lock once per request. Per worker, a request takes $4/192.5\,\text{k} = 20.8$ μs with threads against $4/292.9\,\text{k} = 13.7$ μs with processes, so each request loses about 7 μs, several times the handler's own cost, to obtaining the GIL. The finding is therefore stronger than

Table 7: Transport path, 13-byte body: throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$) at N workers; scaling $S_4 = \bar{x}_4/\bar{x}_1$; busy server cores C_N; throughput per busy core at four workers relative to one, $E_4 = (\bar{x}_4/C_4)/(\bar{x}_1/C_1)$; 99th-percentile latency at four workers.

Build	Engine	Workers as	N=1	N=2	N=3	N=4	S_4	C_1	C_4	E_4	p99$_4$ (ms)
GIL	`uringpy`	thread	140.7 ± 3.0	228.3 ± 6.0	286.5 ± 3.9	328.4 ± 0.8	2.33 ± 0.05	1.18	3.91	0.70	1.7
GIL	`uringpy`	process	144.4 ± 3.8	230.4 ± 7.0	288.5 ± 1.9	328.9 ± 1.2	2.28 ± 0.06	1.19	3.92	0.69	1.6
GIL	`asyncio`	thread	66.8 ± 1.1	47.8 ± 0.8	35.5 ± 0.1	34.4 ± 0.2	0.52 ± 0.01	1.07	1.76	0.31	15.4
GIL	`asyncio`	process	68.1 ± 1.0	116.8 ± 1.4	151.7 ± 2.7	179.6 ± 0.8	2.64 ± 0.04	1.08	3.91	0.73	2.7
GIL	`asyncio-proto`	thread	93.6 ± 1.6	95.2 ± 1.6	59.9 ± 0.5	45.0 ± 0.9	0.48 ± 0.01	1.12	2.17	0.25	16.7
GIL	`asyncio-proto`	process	94.3 ± 1.6	157.6 ± 4.2	201.2 ± 3.4	235.3 ± 1.7	2.50 ± 0.05	1.12	3.90	0.72	3.5
GIL	`uvloop`	thread	72.5 ± 1.0	–	–	43.0 ± 0.5	0.59 ± 0.01	1.09	1.69	0.38	19.9
GIL	`uvloop`	process	72.4 ± 1.9	–	–	193.6 ± 1.9	2.67 ± 0.07	1.09	3.92	0.74	4.2
GIL	`uvloop-proto`	thread	98.0 ± 2.3	–	–	35.0 ± 0.2	0.36 ± 0.01	1.13	1.59	0.25	26.6
GIL	`uvloop-proto`	process	96.5 ± 1.8	–	–	247.0 ± 0.6	2.56 ± 0.05	1.12	3.90	0.74	3.4
free-thr.	`uringpy`	thread	142.4 ± 5.1	229.7 ± 3.7	285.8 ± 5.3	327.0 ± 1.5	2.30 ± 0.08	1.19	3.91	0.70	1.8
free-thr.	`uringpy`	process	142.2 ± 5.1	228.0 ± 3.6	287.8 ± 2.3	325.2 ± 1.6	2.29 ± 0.08	1.19	3.92	0.69	1.7
free-thr.	`asyncio`	thread	63.2 ± 1.8	108.8 ± 0.6	141.0 ± 2.5	166.6 ± 0.9	2.64 ± 0.08	1.08	3.93	0.73	2.8
free-thr.	`asyncio`	process	64.9 ± 1.1	111.0 ± 1.6	146.1 ± 2.4	172.5 ± 2.6	2.66 ± 0.06	1.08	3.92	0.73	2.8
free-thr.	`asyncio-proto`	thread	90.0 ± 1.8	147.2 ± 2.8	189.8 ± 1.7	224.6 ± 1.2	2.50 ± 0.05	1.12	3.90	0.72	3.7
free-thr.	`asyncio-proto`	process	91.3 ± 1.1	151.7 ± 1.7	196.1 ± 1.5	230.4 ± 1.3	2.52 ± 0.03	1.12	3.92	0.72	3.6

`uvloop` was measured in a separate experiment at one and four workers only and is not installed in the free-threaded image.

Table 8: Application-handler workload: every request calls a Python handler. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), scaling, busy server cores and 99th-percentile latency at four workers.

Build	Engine	Workers as	N=1	N=2	N=4	S_2	S_4	C_4	p99$_4$ (ms)
GIL	`uringpy-app`	thread	122.3 ± 3.0	189.5 ± 5.8	192.5 ± 7.2	1.55 ± 0.06	1.57 ± 0.07	3.25	3.5
GIL	`uringpy-app`	process	124.2 ± 3.7	200.3 ± 5.7	292.9 ± 1.4	1.61 ± 0.07	2.36 ± 0.07	3.91	1.8
GIL	`asyncio-app`	thread	60.3 ± 1.1	39.9 ± 1.4	30.5 ± 0.4	0.66 ± 0.03	0.51 ± 0.01	1.68	17.2
GIL	`asyncio-app`	process	59.9 ± 1.7	102.8 ± 2.2	155.4 ± 10.9	1.72 ± 0.06	2.60 ± 0.20†	3.93	3.9
free-thr.	`uringpy-app`	thread	123.9 ± 3.9	199.9 ± 3.1	286.2 ± 10.6	1.61 ± 0.06	2.31 ± 0.11	3.89	2.0
free-thr.	`uringpy-app`	process	124.3 ± 1.7	200.5 ± 4.6	283.1 ± 16.4	1.61 ± 0.04	2.28 ± 0.14	3.91	3.2
free-thr.	`asyncio-app`	thread	55.7 ± 0.9	95.7 ± 0.9	149.4 ± 1.1	1.72 ± 0.03	2.68 ± 0.05	3.91	3.2
free-thr.	`asyncio-app`	process	56.6 ± 1.2	98.5 ± 1.0	154.6 ± 1.5	1.74 ± 0.04	2.73 ± 0.06	3.91	3.2

† A configuration in this row had a coefficient of variation above 5%.

the model suggests: a handler of about one microsecond, entered once per request, already costs a third of four-thread throughput. **[[PENDING: Handler sweep: add the measured hold time t_p and wait time w per request at each N, with and without added handler work, and state how much of the 7 μs is measured wait. A prediction must use one-worker quantities and a hand-off cost fitted at one point and checked at others.]]**

The comparison with `asyncio` in Table 8 uses its streams API, which Table 7 shows to be the slower of its two. With the same handler, `asyncio` threads fall to 0.51× of one worker. `uringpy-app` with four threads is 1.24 ± 0.10 times faster than `asyncio-app` with four processes, and process against process it is 1.88 ± 0.13 times faster. On the free-threaded build the plateau disappears for both: `uringpy-app` threads scale by 2.31 ± 0.11, the same as its processes, and deliver 1.49× their throughput on the GIL build; `asyncio-app` threads scale by 2.68 ± 0.05; and the gap between the two at four workers is 1.92 ± 0.07. **[[PENDING: Add Protocol-API and `uvloop` handler baselines. From the transport results, four Protocol-API processes with this handler are likely to match or exceed `uringpy-app`'s four threads under the GIL; report that plainly if so.]]**

Table 9: Response-body size (four worker threads, GIL build). Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$); for `uringpy` also the data rate received by the client, busy server cores and system calls per request.

Body	`uringpy`	`asyncio`	Gbit/s	C	sys/req
64 B	328.1 ± 2.2	34.2 ± 0.4	0.4	3.92	0.02
1 KiB	321.3 ± 0.8	33.8 ± 0.5	2.9	3.92	0.02
16 KiB	119.4 ± 0.2	29.6 ± 0.2	15.7	3.31	0.58
64 KiB	30.0 ± 0.2	27.3 ± 0.2	15.8	1.37	0.66
256 KiB	7.48 ± 0.04	7.50 ± 0.01	15.7	1.21	1.09
1 MiB	1.86 ± 0.01	1.86 ± 0.01	15.7	1.59	2.50

Response size. Table 9 varies the response body at four worker threads on the GIL build. Up to 1 KiB throughput changes by 2%. From 16 KiB upward the client receives 15.7–15.8 Gbit/s whatever the body size, and from 256 KiB upward it receives that from either engine: the network path is the limit, and those rows mea-

sure it and not the servers. `uringpy` reaches the limit at 16 KiB with 3.31 cores busy, and at larger sizes keeps 1.2–1.6 cores busy. As responses grow, each is sent in several pieces and batches shrink, so its system calls per request rise from 0.02 to 2.5. The runtime's advantage in requests per second is thus confined to responses small enough for per-request overhead, not data transfer, to dominate: on this network `uringpy` is limited by the path from 16 KiB, and the two engines are level from 256 KiB. We did not determine what sets the limit of the path. The baseline in this table is `asyncio` in thread mode, so the difference at small sizes combines the two effects separated above.

6 Discussion

Attributing a speed-up. A Python loop on `io_uring` that is faster than `asyncio` has not thereby shown that `io_uring` is the reason. In our measurements the interface is worth a factor of 1.16–1.26 on one worker, and the benefit of batching is exhausted at 16 completions per call, so a figure of several hundred completions per system call, though real, says little about throughput. Most of the distance between one `asyncio` worker and one `uringpy` worker (2.11×) comes from what the loop does per event, not from the interface it waits on.

When C-side reaping helps. Three regimes emerge. On a GIL build, with requests that need no Python, it is the difference between threads that scale and threads that lose throughput, and it gives one interpreter the capacity of a process per core. On a free-threaded build it is a constant factor of 1.5–2×, because the baseline scales too. With Python on every request, under the GIL, its benefit for threads is limited by the cost of acquiring the GIL once per request, which in our measurement is several times the cost of the handler itself, and processes or the free-threaded build are needed to use all cores.

Threads or processes. Since worker processes reach the same throughput as `uringpy`'s worker threads, the case for threads is not speed. It is that the workers live in one interpreter and can share state without inter-process communication. We did not measure a workload that depends on shared state, so this paper establishes that the option exists at no cost in throughput, not how much a given application gains from it.

7 Threats to Validity

Table 10 lists the ways a measurement of this kind can mislead and what was done about each. The limits that remain are these.

Testbed. The server has four cores, so scaling beyond four workers is not shown, and at four workers the kernel's network processing competes with the workers for the same cores; E_4 in Table 7 quantifies that. The machines are cloud virtual machines of one family running one kernel and one CPython version (3.14.7), and the free-threaded results describe that version of an implementation that is still changing. Above 16 KiB responses the network path, not the server, was the limit.

Workload. The workloads are one protocol exchange with a fixed or lightly computed response; requests are not parsed, and TLS, request bodies and heavier application logic are not covered. The transport workload runs no Python in `uringpy` and Python on every event in the baselines, which is the most favourable case for the runtime. Load is 400 keep-alive connections from a closed-loop generator and nothing else: the value of batching depends on how many completions are waiting, so the interface effect may be smaller, or reversed, under light load, as the batch cap of one suggests. `io_uring` is used in its basic form (Section 4); features such as multishot receive or deferred task running were not evaluated. The `asyncio` and `uvloop` transports set `TCP_NODELAY`; the C reactors and the Python loops do not, which we expect to be immaterial for responses of one segment but did not test. The Python loops of the 2×2 design are bare loops, so the loop-placement effect measured on one worker is a lower bound for a full event loop. Latency is measured by a closed-loop generator and is not a tail-latency estimate. The latency columns are means of per-run 99th percentiles; for `py-uring` the five runs range from 0.8 to 1.25 s.

Comparisons across experiments. Repetitions are interleaved within an experiment, not between experiments. One `uringpy` worker was measured in three of them, at 140.7 ± 3.0, 146.1 ± 5.4 and 144.0 ± 4.5 thousand requests per second, a spread of 4%. Ratios between configurations of one experiment are unaffected. Comparisons that span two experiments carry that additional uncertainty, and their intervals, where given, reflect only the variation within each: those involving `uvloop`, the capped `io_uring` loop against `epoll`, `py-epoll` against `asyncio`, the cost of the handler on one worker, and every comparison of the GIL build with the free-threaded build. The same configuration, `uringpy` threads on the GIL build, has $S_4 = 2.33 \pm 0.05$ in Table 7 and 2.24 ± 0.08 in Table 4; the four-worker means agree and the one-worker means differ. All runs were made on one pair of machines in one day.

8 Conclusion

For a Python server on `io_uring`, the system-call interface and the placement of the per-event loop are separate levers with separate effects. In our measurements the interface, and batching up to about 16 completions per call, set the throughput of a single worker, by a factor of 1.16–1.26; the placement of the loop decides whether worker threads scale. A runtime that reaps completions and runs the connection protocol in C with the GIL released gives one interpreter on a standard build the throughput of a process per worker—2.33× one worker on four cores, where `asyncio` and `uvloop` threads fall below one

Table 10: Pitfalls and how each is handled.

Pitfall	What is done	Evidence
The load generator, not the server, is the limit	The client has twice the server's cores; client processor use is recorded for every run and configurations above 85% are flagged	Section 5.5; raw data
Hyper-threads counted as cores	Machine family with one vCPU per physical core	Table 2
Noise mistaken for an effect	Five repetitions, interleaved in shuffled order, a fresh server per run, 95% confidence intervals; cross-experiment spread stated	every results table
A weak baseline	`asyncio` through both its streams and its faster Protocol API; `uvloop`; one interpreter version for every engine	Table 3
GIL and free-threaded builds differing in more than the GIL	Both interpreters built from one recipe; the GIL state reported by the server is recorded per run	Table 2
Crediting `io_uring` for what C code does, or the reverse	The 2×2 design varies the two separately; the batch cap varies batching alone	Tables 4–6
System-call counts estimated, not measured	Both C reactors count their system calls and completed requests	Tables 4, 6
A one-worker baseline that also uses other cores	Server processor use recorded; busy cores and throughput per busy core reported next to scaling	Table 7
A model fed with assumed numbers	GIL hold and wait time measured per request	handler results
Engines doing different protocol work	Every engine answers each `recv` with identical bytes; none parses HTTP	Section 4
Errors hidden in a throughput figure	Socket errors, non-2xx responses, handler errors and failed runs recorded and flagged	raw data
Numbers not traceable to code	Commit, image, interpreter and seed recorded per experiment; raw runs archived; summaries regenerable	repository

worker—and exceeds the best process-per-worker baseline by 1.33×. The advantage has limits, which we measured. On the free-threaded build `asyncio` threads scale as well and a factor of 1.5–2× in speed remains. With a Python handler on every request, thread scaling under the GIL stops at 1.57×, and processes or the free-threaded build are needed to use every core. With responses of 16 KiB or more, the network and not the server was the limit. The GIL, not the system call, governs scaling; the system call still accounts for part of the speed of one worker.

Acknowledgment

[[PENDING: Author to confirm the wording before submission. IEEE requires content generated by AI to be disclosed in this section.]] The author used Claude (Anthropic), an AI assistant, in revising this work: to review an earlier draft and propose additional experiments in response; to write parts of the implementation (the application-handler reactor, the `epoll` reactor, the benchmark driver, the table generator and the tests) and of the documentation; and to draft text in every section of this paper, including the abstract. The author ran all experiments, reviewed and tested this content, and is responsible for the paper's claims.

Code and Data Availability

The runtime and all benchmark scripts are open source (MIT) at `https://github.com/marimuthuvm/uringpy` and released on PyPI (`pip install uringpy`). The repository contains the raw output of every run reported here (`benchmarks/results/`), the script that generates the result tables from it (`benchmarks/make_tables.py`), the container recipe for both interpreters, and a description of each experiment (`benchmarks/EXPERIMENTS.md`). **[[PENDING: Archive the release that contains these results and cite its DOI here; the earlier archive, `https://doi.org/10.5281/zenodo.23129560`, predates them.]]**

Conflict of Interest

The author has no financial or personal relationships that could have influenced this work, and declares no conflict of interest. No external funding was received.

References

[1] J. Axboe, "Efficient IO with io_uring," 2019. [Online]. Available: `https://kernel.dk/io_uring.pdf`

[2] MagicStack, "uvloop: Ultra fast asyncio event loop." [Online]. Available: `https://github.com/MagicStack/uvloop`

[3] L. Soares and M. Stumm, "FlexSC: Flexible system call scheduling with exception-less system calls," in *Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2010.

[4] S. Han, S. Marinos, S. Shenker, and S. Ratnasamy, "MegaPipe: A new programming interface for scalable network I/O," in *Proc. 10th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2012.

[5] E. Jeong, S. Woo, M. Jamshed, H. Jeong, S. Ihm, D. Han, and K. Park, "mTCP: A highly scalable

user-level TCP stack for multicore systems," in *Proc. 11th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2014.

[6] A. Belay, G. Prekas, A. Klimovic, S. Grossman, C. Kozyrakis, and E. Bugnion, "IX: A protected dataplane operating system for high throughput and low latency," in *Proc. 11th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2014.

[7] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage APIs: A systematic study of libaio, SPDK, and io_uring," in *Proc. 15th ACM Int. Systems and Storage Conf. (SYSTOR)*, 2022, doi: 10.1145/3534056.3534945.

[8] M. Jasny, M. El-Hindi, T. Ziegler, V. Leis, and C. Binnig, "High-performance DBMSs with io_uring: When and how to use it," *Proc. VLDB Endow.*, vol. 19, no. 9, 2026.

[9] G. Banga, J. C. Mogul, and P. Druschel, "A scalable and explicit event delivery mechanism for UNIX," in *Proc. USENIX Annu. Technical Conf.*, 1999.

[10] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable Internet services," in *Proc. 18th ACM Symp. Operating Systems Principles (SOSP)*, 2001.

[11] G. van Rossum, "PEP 3156: Asynchronous IO support rebooted: the asyncio module," Python Enhancement Proposals, 2012.

[12] A. Pesterev, J. Strauss, N. Zeldovich, and R. T. Morris, "Improving network connection locality on multicore systems," in *Proc. 7th ACM European Conf. Computer Systems (EuroSys)*, 2012.

[13] M. Kerrisk, "The SO_REUSEPORT socket option," *LWN.net*, Mar. 2013. [Online]. Available: `https://lwn.net/Articles/542629/`

[14] P. Enberg, A. Rao, and S. Tarkoma, "The impact of thread-per-core architecture on application tail latency," in *Proc. ACM/IEEE Symp. Architectures for Networking and Communications Systems (ANCS)*, 2019.

[15] D. Beazley, "Understanding the Python GIL," presented at PyCon, Atlanta, GA, USA, 2010. [Online]. Available: `http://www.dabeaz.com/GIL/`

[16] R. Odaira, J. G. Castanos, and H. Tomari, "Eliminating global interpreter locks in Ruby through hardware transactional memory," in *Proc. 19th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2014, doi: 10.1145/2555243.2555247.

[17] S. Gross, "PEP 703: Making the Global Interpreter Lock Optional in CPython," Python Enhancement Proposals, 2023.

[18] T. Wouters, M. Page, and S. Gross, "PEP 779: Criteria for supported status for free-threaded Python," Python Enhancement Proposals, 2025.

[19] E. Snow, "PEP 684: A Per-Interpreter GIL," Python Enhancement Proposals, 2022.

[20] E. Snow, "PEP 734: Multiple Interpreters in the Stdlib," Python Enhancement Proposals, 2023.

[21] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 31–39, 2011.

[22] K. Aditya, "Scaling asyncio on free-threaded Python," Quansight Labs, Sep. 2025. [Online]. Available: `https://labs.quansight.org/blog/scaling-asyncio-on-free-threaded-python`

[23] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.

[24] "uringloop: An io_uring event loop for asyncio." [Online]. Available: `https://github.com/thearchitector/uringloop`

[25] "uringcore: High-performance asyncio event loop using io_uring." [Online]. Available: `https://github.com/ankitkpandey1/uringcore`

[26] "Granian: A Rust HTTP server for Python applications." [Online]. Available: `https://github.com/emmett-framework/granian`

[27] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007, doi: 10.1145/1297027.1297033.

[28] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!," in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009, doi: 10.1145/1508244.1508275.

[29] A. Uta, A. Custura, D. Duplyakin, I. Jimenez, J. Rellermeyer, C. Maltzahn, R. Ricci, and A. Iosup, "Is big data performance reproducible in modern cloud networks?," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020.

[30] W. Glozer, "wrk: Modern HTTP benchmarking tool." [Online]. Available: `https://github.com/wg/wrk`

[31] G. Tene, "wrk2: A constant throughput, correct latency recording variant of wrk." [Online]. Available: `https://github.com/giltene/wrk2`

[32] S. Eyerman and L. Eeckhout, "Modeling critical sections in Amdahl's law and its implications for multicore design," in *Proc. 37th Annu. Int. Symp. Computer Architecture (ISCA)*, 2010.

[33] Google Cloud, "CPU platforms," Compute Engine documentation. [Online]. Available: `https://docs.cloud.google.com/compute/docs/cpu-platforms`

Marimuthu Velayutham is an independent researcher working on high-performance systems software and language runtimes, with interests in asynchronous I/O, operating-system interfaces, and the performance of managed-language runtimes.